UNIVERSIDAD CHAMPAGNAT - PROGRAMACION DISTRIBUIDA

Sistema Distribuido de Venta de Entradas

Arquitectura de Microservicios, Concurrencia en Redis, Docker y Seguridad JWT

Proyecto integrador (TP1 a TP7): Evolucion de una API REST monolitica hacia una arquitectura de microservicios contenerizados con Docker Compose, persistencia independiente en MongoDB, exclusion mutua distribuida en Redis mediante Scripts Lua y autenticacion/autorizacion stateless con JWT (HS256) y RBAC.

1. User Service (:3000)

Integrante: Marcos

- Gestion de identidad y registro
- Hashing con bcryptjs (10 rondas)
- Emision de JWT (HS256)
- Propiedad del recurso y RBAC
- Base propia MongoDB (users-db)

2. Inventory Service (:3001)

Integrante: Nahuel

- Propietario del catalogo y stock
- Encapsula el Lock en Redis (SET NX PX)
- Liberacion atomica con Script Lua
- Decremento atomico en inventory-db
- Control de acceso RBAC (ADMIN)

3. Checkout Service (:3002)

Integrante: Lisandro

- Orquestador stateless de compras
- Propagacion de JWT hacia Inventory
- Timeout de 3000ms (AbortController)
- Manejo de fallos parciales (503)
- Suite de pruebas reproducibles

Stack: Node.js 20 LTS | Express | MongoDB 7 | Redis 7 | JWT (HS256) | Winston | Swagger OpenAPI 3.0 | Docker Compose

1. Objetivo del Proyecto y Evolucion Arquitectonica (TP1 a TP7)

EVOLUCION

El Problema de Dominio: Venta Concurrente de Entradas con Stock Limitado

En la venta de entradas para eventos de alta demanda, multiples usuarios intentan adquirir simultaneamente las ultimas localidades disponibles. Sin mecanismos adecuados de exclusion mutua y desacoplamiento, se producen condiciones de carrera (race conditions), sobreventa (stock negativo) y fallos en cascada.

Nuestro sistema evoluciona desde una API monolitica hasta una arquitectura de 3 microservicios independientes que garantizan consistencia del stock, seguridad distribuida y tolerancia a fallos.

Fase 1: Base Monolitica (TP1 - TP3)

- TP1 (Servidor Seguro): API REST con Node.js y Express, variables de entorno (.env) y cifrado en transito HTTPS.
- TP2 (Exclusion Mutua en Redis): Implementacion de cerrojo distribuido (SET NX PX 5000) y liberacion atomica mediante Script Lua para evitar condiciones de carrera.
- TP3 (Persistencia y Observabilidad): Base de datos MongoDB con Mongoose, logging estructurado con rotacion diaria en Winston y documentacion interactiva en Swagger UI.

Fase 2: Microservicios y Docker (TP5 - TP6)

- TP5 (Diseno DDD): Descomposicion del monolito en User Service, Inventory Service y Checkout Service. Definicion de limites, contratos REST y propiedad de datos.
- TP6 (Contenedores y Resiliencia): Imagenes Docker individuales, orquestacion con Docker Compose, red interna backend-net, volumenes persistentes y healthchecks.
- Encapsulacion del Lock: Inventory Service centraliza tanto el stock en MongoDB como el Lock en Redis.

Fase 3: Seguridad JWT y RBAC (TP7)

- Registro y Login Seguro: Almacenamiento de contrasenas con hashing bcryptjs (10 rondas) y emision de JWT firmado con HS256 en user-service.
- Verificacion Stateless: Cada microservicio valida autonomamente la firma y expiracion del token sin consultar a user-service.
- Propagacion y RBAC: Checkout propaga el token JWT hacia Inventory; control estricto de roles (USER vs ADMIN) y propiedad del recurso.

2. Arquitectura General de Microservicios y Propiedad de Datos

ARQUITECTURA

Topologia del Sistema en Docker Compose (Red Interna Aislada: backend-net)

user-service (:3000)
Identidad, Bcrypt, Login JWT y RBAC
BD propia: MongoDB (users-db)
Ca = 1 | Ce = 0 | Inestabilidad I = 0.00

checkout-service (:3002)
Orquestador de Compras (Stateless)
Timeout 3000ms + Propaga JWT
Ca = 1 | Ce = 1 | Inestabilidad I = 0.50

PUT /reducir
---------->

inventory-service (:3001)
Catalogo, Stock y Concurrencia
Redis (Locks) + MongoDB (Stock)
Ca = 2 | Ce = 0 | Inestabilidad I = 0.00

Principio de Propiedad de Datos (Database per Service)

- Usuarios (MongoDB tp6_usuarios): Propiedad exclusiva de user-service. Ningun otro servicio accede a esta base.
- Catalogo y Stock (MongoDB tp6_inventario): Propiedad exclusiva de inventory-service.
- Locks Distribuidos de Stock (Redis inventory-redis): Propiedad interna exclusiva de inventory-service.
- Contexto de Sesion (Stateless JWT): Emitido por user-service y verificado criptograficamente por todos los servicios.

Bajo Acoplamiento y Alta Cohesion

- Desacoplamiento de Identidad: Gracias a la firma simetrica JWT (HS256), ni Checkout ni Inventory necesitan consultar a User Service durante una compra.
- Desacoplamiento de Concurrencia: Checkout desconoce la existencia de Redis; delega la reduccion de stock a Inventory a traves de su interfaz publica REST.
- Independencia de Despliegue: User Service e Inventory Service poseen acoplamiento eferente nulo (Ce = 0), logrando maxima estabilidad arquitectonica (I = 0.00).

3. Microservicio 1: User & Identity Service (Puerto 3000)

USER SERVICE

Responsabilidades y Reglas de Negocio

- Registro de Usuarios (POST /auth/register): Valida campos obligatorios, longitud minima de contrasena (>= 6 caracteres) y unicidad de DNI y correo electronico (retorna 409 Conflict ante duplicados).
- Hashing Criptografico: Aplica bcrypt.hash(password, 10) antes de persistir en MongoDB. El esquema Mongoose elimina automaticamente passwordHash en el metodo toJSON.
- Autenticacion y Emision JWT (POST /auth/login): Verifica credenciales y cuenta activa, emitiendo un token firmado con HS256 que incluye sub (ID), role (USER/ADMIN), iss, iat y exp.

Control de Acceso (RBAC) y Propiedad del Recurso

- Listado Global (GET /api/usuarios): Restringido exclusivamente al rol ADMIN (retorna 403 Forbidden a usuarios estandar).
- Propiedad del Recurso en Perfil (GET /api/usuarios/:id): Un usuario con rol USER solo puede consultar su propio documento (req.user.sub === req.params.id). Un ADMIN puede consultar cualquier perfil.
- Baja de Cuentas (DELETE /api/usuarios/:id): Operacion critica exclusiva para el rol ADMIN.

Estructura del Token JWT Emitido y Persistencia en users-db

Payload firmado en POST /auth/login y verificado de forma autonoma por los 3 microservicios:

```
const payload = { sub: usuario._id.toString(), role: usuario.role, iss: 'user-service' };
```

```
const accessToken = jwt.sign(payload, JWT_SECRET, { algorithm: 'HS256', expiresIn: JWT_EXPIRES_IN });
```

Disponibilidad ante caidas: Si user-service se detiene luego de haber emitido el token, checkout-service e inventory-service continuan procesando compras normalmente porque validan el JWT en memoria.

4. Microservicio 2: Inventory & Stock Service (Puerto 3001)

INVENTORY SERVICE

Propiedad del Stock y Encapsulacion de la Concurrencia

- Catalogo Publico (GET /api/inventario): Permite consultar los eventos y la cantidad de entradas disponibles sin requerir autenticacion.
- Gestion Administrativa de Stock (POST /api/inventario/stock): Permite crear productos o actualizar existencias (upsert). Protegido por RBAC exclusivamente para el rol ADMIN.
- Reduccion Concurrente (PUT /api/inventario/reducir): Encapsula el Lock distribuido en Redis y el decremento atomico en MongoDB para garantizar stock >= 0.

Por que el Lock de Redis vive en Inventory Service?

- Coherencia con el Dominio: Inventory es la unica autoridad sobre el stock; ninguna capa externa debe coordinar bloqueos sobre sus datos.
- Prevencion de Condiciones de Carrera: Al recibir peticiones simultaneas, la primera adquiere la clave en Redis con NX (Not Exists) y TTL de 5000ms; las concurrentes reciben 409 Conflict de inmediato.
- Liberacion Segura en finally: El Script Lua verifica que el UUID coincida con el propietario antes de borrar el lock.

Implementacion de la Seccion Critica en inventory-service/index.js

1. Adquisicion del Lock exclusivo en Redis con expiracion automatica de 5 segundos:

```
lockAdquirido = await redisClient.set(lockKey, ownerId, { NX: true, PX: 5000 }); // Si es null -> 409 Conflict
```

2. Decremento atomico en MongoDB garantizando que el stock sea mayor a cero:

```
const item = await Inventario.findOneAndUpdate({ producto: nombreProd, stock: { $gt: 0 } }, { $inc: { stock: -1 } }, { new: true });
```

3. Liberacion atomica en bloque finally mediante Script Lua:

```
if redis.call('get', KEYS[1]) == ARGV[1] then return redis.call('del', KEYS[1]) else return 0 end
```

5. Microservicio 3: Checkout & Transaction Service (Puerto 3002)

CHECKOUT SERVICE

Orquestacion de Compras y Propagacion de Identidad

- Fachada Transaccional (POST /api/comprar): Recibe la solicitud de compra del cliente autenticado (rol USER o ADMIN).
- Diseno 100% Stateless: No posee base de datos propia ni conexion a Redis, permitiendo escalar multiples instancias sin estado local.
- Propagacion del Contexto de Seguridad: Extrae el token JWT verificado y lo reenvia en el header Authorization: Bearer <token> hacia inventory-service para mantener trazabilidad de extremo a extremo.

Timeout Explicito y Semantica de Errores HTTP

- Control de Tiempo de Espera (3000 ms): Utiliza AbortController para cancelar llamadas lentas hacia Inventory y evitar que Checkout quede bloqueado indefinidamente.
- Traduccion Semantica de Respuestas:
  * 200 OK -> Compra confirmada con stockRestante.
  * 409 Conflict -> Recurso bloqueado por concurrencia en Inventario.
  * 400 Bad Request -> Stock agotado (stock === 0).
  * 404 Not Found -> Producto inexistente.
  * 503 Service Unavailable -> Timeout o caida de Inventory Service.

Comunicacion HTTP Interna con Timeout y Propagacion en checkout-service/index.js

Invocacion remota usando fetch nativo y variables de entorno (INVENTORY_SERVICE_URL):

```
const controller = new AbortController(); const timeoutId = setTimeout(() => controller.abort(), 3000);
const response = await fetch(`${INVENTORY_SERVICE_URL}/api/inventario/reducir`, {
  method: 'PUT', headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer ${req.rawToken}` },
  body: JSON.stringify({ producto }), signal: controller.signal });
```

6. Analisis de Fallos Parciales y Resiliencia en el Flujo de Compra

FALLOS PARCIALES

Secuencia del Escenario de Fallo Parcial Distribuido

1. Participantes: Cliente -> checkout-service -> inventory-service (Redis + MongoDB).

2. Cambios de Estado: Inventory adquiere el lock en Redis, descuenta exitosamente el stock en MongoDB (pasa de 1 a 0) y libera el lock.

3. Punto de Falla: Se produce un corte de red o demora antes de que la respuesta HTTP 200 OK llegue desde Inventory hacia Checkout.

4. Respuesta al Cliente: Checkout aborta la espera al cumplirse los 3000ms de Timeout y devuelve HTTP 503 Service Unavailable.

5. Inconsistencia Resultante: El stock ya fue descontado en la base de datos de inventario, pero el cliente recibio un error.

Mecanismos de Recuperacion y Compensacion Propuestos

- 1. Claves de Idempotencia (Idempotency-Key): El cliente genera un identificador unico UUID por intento de compra. Si reintenta tras recibir 503, Inventory reconoce que ese UUID ya desconto stock y devuelve la confirmacion previa sin volver a restar.

- 2. Patron Saga (Transacciones Compensatorias): Ante un fallo confirmado en la orquestacion, Checkout emite una orden compensatoria (POST /api/inventario/reponer) para revertir el descuento.

- 3. Brokers de Mensajeria (RabbitMQ / Kafka): Reemplazar o complementar llamadas sincronicas con eventos durables y confirmaciones ACK.

Aislamiento de Fallos: Prueba de Indisponibilidad de Dependencias

- Si el contenedor de inventory-service se detiene, checkout-service NO colapsa ni finaliza su proceso Node.js.

- Captura el error de conexion o timeout, registra el evento con nivel ERROR en Winston detallando la dependencia afectada y responde de manera controlada HTTP 503 Service Unavailable.

- En cuanto inventory-service vuelve a estar activo, el flujo de compras se recupera automaticamente sin reiniciar checkout-service.

7. Contenerizacion e Infraestructura con Docker Compose

DOCKER COMPOSE

1. Imagenes y .dockerignore

- Imagen base liviana: node:20-alpine en los 3 microservicios.
- Instalacion limpia de dependencias de produccion: npm install --omit=dev.
- Archivos .dockerignore en cada servicio que excluyen node_modules, .env, *.log, logs/, *.pfx y .git.

2. Red Interna y Seguridad

- Red bridge privada (backend-net): Resolucion DNS interna por nombre de servicio (users-db, inventory-db, redis, inventory-service).
- Puertos publicados al host: Solo 3000, 3001 y 3002.
- Bases MongoDB y Redis ocultas al exterior (sin puertos expuestos al host).

3. Volumenes y Healthchecks

- Persistencia real fuera del ciclo de vida del contenedor mediante volumenes nombrados: users_mongo_data e inventory_mongo_data.
- Orden de arranque controlado con healthchecks y depends_on: condition: service_healthy en los 6 contenedores.

Configuracion por Variables de Entorno: Ejecucion Local vs. Docker Compose

El mismo codigo fuente funciona sin cambios tanto en local como en contenedores mediante configuracion externalizada (.env.example):

```
Entorno Docker Compose -> MONGODB_URI=mongodb://inventory-db:27017/tp6_inventario | REDIS_URL=redis://redis:6379
Entorno Local Node.js  -> MONGODB_URI=mongodb://127.0.0.1:27017/tp6_inventario    | REDIS_URL=redis://127.0.0.1:6379
Comando unico de despliegue: docker compose up -d --build
```

8. Matriz RBAC, Logging con Winston y Documentacion OpenAPI

RBAC & SWAGGER

Matriz de Control de Acceso Basado en Roles (RBAC - TP7)

- User Service (:3000):
  * GET /health, POST /auth/register, POST /auth/login -> Publicos.
  * GET /api/usuarios/:id -> Autenticado (Propio USER sub === :id o ADMIN).
  * GET /api/usuarios y DELETE /api/usuarios/:id -> Exclusivo ADMIN.
- Inventory Service (:3001):
  * GET /health, GET /api/inventario -> Publicos.
  * POST /api/inventario/stock -> Exclusivo ADMIN (carga/modificacion de stock).
  * PUT /api/inventario/reducir -> Autenticado (USER o ADMIN con token propagado).
- Checkout Service (:3002):
  * GET /health -> Publico | POST /api/comprar -> Autenticado (USER o ADMIN).

Observabilidad y Documentacion

- Logging Estructurado (Winston):
  * Registra timestamp, nivel (INFO, WARN, ERROR), nombre del microservicio y metadatos de contexto (userId, role, stockRestante).
  * Salida por stdout/stderr consultable con docker compose logs -f.
- Portales OpenAPI 3.0 (Swagger UI):
  * http://localhost:3000/api-docs
  * http://localhost:3001/api-docs
  * http://localhost:3002/api-docs

Higiene del Repositorio y Buenas Practicas de Seguridad

- Sin secretos ni archivos autogenerados en Git: Se retiraron del versionado los certificados criptograficos (.pfx) y archivos de logs (.log y *-audit.json).
- Archivos .gitignore y .dockerignore configurados en la raiz y en cada microservicio para proteger credenciales y mantener imagenes limpias.
- Plantilla .env.example documentada con todas las variables requeridas por los tres servicios.

9. Suite de Pruebas Reproducibles Automatizadas y Conclusiones

VALIDACION Y TESTS

Ejecucion Automatizada de Punta a Punta (npm test / node tests/test-reproducible.js)

- Fase 1 (Healthchecks): Verifica que user-service (:3000), inventory-service (:3001 con MongoDB y Redis conectados) y checkout-service (:3002) respondan 200 OK.
- Fase 2 (Identidad y JWT): Registra un usuario ADMIN y un usuario USER en user-service y obtiene sus tokens JWT firmados con HS256.
- Fase 3 (Preparacion de Estado Inicial): El ADMIN fija el stock de 'Entrada Conferencia' en exactamente 1 unidad mediante POST /api/inventario/stock.
- Fase 4 (Prueba de Concurrencia Real con Promise.all): Dispara 2 peticiones POST /api/comprar simultaneas compitiendo por la unica entrada disponible. Comprueba que exactamente 1 obtiene 200 OK y la otra es rechazada (409 Conflict / 400 Bad Request), dejando el stock final en 0 sin sobreventa.
- Fase 5 y 6 (Stock Agotado, RBAC y Propiedad del Recurso): Verifica el rechazo 400 con stock en cero, el bloqueo 403 cuando un USER intenta alterar stock o leer un perfil ajeno, y el rechazo 401 ante peticiones sin token o con token adulterado.

Comandos para Reproducir la Demostracion

1. Levantar la arquitectura completa:

```
docker compose up -d --build
```

2. Ejecutar la suite automatizada de 16 pruebas:

```
npm test
```

3. Pruebas manuales interactivas:

Archivo tests/pruebas-api.http listo para ejecutar en VS Code.

Conclusiones del Proyecto

- Limites de dominio claros y responsabilidades bien definidas.
- Concurrencia encapsulada en el propietario del dato (Inventory Service) combinando Locks en Redis + Scripts Lua y MongoDB.
- Seguridad distribuida autonoma con JWT, Bcrypt y RBAC.
- Entorno 100% reproducible y verificado mediante contenedores Docker y pruebas automatizadas.